

FULL STACK DEVELOPMENT

FASE 2 | AULA 05 - CRIAÇÃO DE APIS RESTFUL COM EXPRESS.JS.

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EXEMPLO

O QUE VEM POR AÍ?

A API Node Express é um conjunto de ferramentas que facilita a construção de APIs RESTful com Node.js e Express.js. Ela fornece funcionalidades para:

- Criar rotas: URLs que mapeiam para funções de manipulação de requisições e respostas.
- Gerenciar requisições: acesso a informações da requisição, como cabeçalhos, parâmetros e corpo.
- Gerar respostas: envio de dados de volta ao cliente em diversos formatos, como JSON, HTML e texto.
- Middleware: funções que interceptam requisições e respostas, permitindo adicionar funcionalidades como autenticação, validação e log.

Talvez você esteja se perguntando: “Por que optar pela Node Express API?”. Bom, há diversas razões para escolher essa tecnologia na construção de APIs, como:

- Fácil de aprender e usar: a curva de aprendizado é suave para iniciantes.
- Desempenho e escalabilidade: alta performance e capacidade de lidar com grande volume de requisições.
- Comunidade vibrante: ampla comunidade de desenvolvedores(as) e recursos disponíveis.

Nesta aula, você aprenderá como:

- Criar uma API Node Express básica;
- Definir rotas para sua API;
- Gerenciar requisições e respostas;
- Usar middleware para adicionar funcionalidades à sua API.

HANDS ON

Nesta seção prática, vamos criar uma API utilizando o Node.js e o framework Express.

Exploraremos o Node.js, que, como vimos anteriormente, é um ambiente de desenvolvimento versátil que permite criar APIs robustas e escaláveis em JavaScript.

Além disso, é importante lembrar que o Express é um framework popular que facilita a criação de APIs RESTful com Node.js. Você aprenderá a configurar rotas, definir métodos HTTP e manipular dados com maestria.

Ao longo da jornada, você colocará em prática o conhecimento obtido e construirá sua própria API passo a passo, consolidando seus conhecimentos e se tornando expert na arte da comunicação entre sistemas.

SAIBA MAIS

REST

Além dos quatro principais métodos HTTP (GET, POST, PUT e DELETE), há vários outros métodos que podem ser utilizados para interagir com recursos na web. Veremos alguns nesta aula:

HEAD

O método HEAD é semelhante ao GET, mas o servidor responde apenas com os cabeçalhos da resposta, sem o corpo da mensagem. Isso é útil quando você deseja obter informações sobre um recurso, como verificar se ele existe ou obter informações sobre o tipo de conteúdo sem baixá-lo.

Este método é utilizado para obter informações sobre os métodos HTTP suportados por um servidor para um determinado recurso.

```
curl -I http://example.com/resource
```

Isso retornará apenas os cabeçalhos da resposta HTTP para o recurso especificado em vez de baixar todo o conteúdo.

OPTIONS

Esse método é usado para obter informações sobre os métodos HTTP suportados por um servidor para um determinado recurso. Ele possibilita determinar quais métodos são permitidos em um recurso, além de obter informações sobre os cabeçalhos CORS (Cross-Origin Resource Sharing) permitidos para uma determinada origem. Exemplo:

```
curl -X OPTIONS http://example.com/resource
```

Isso retornará uma lista de métodos HTTP suportados pelo servidor para o recurso especificado.

PATCH

O PATCH é empregado para aplicar modificações parciais a um recurso. Em vez de substituir completamente o recurso, como o PUT faz, o PATCH é usado para aplicar modificações incrementais ou parciais ao recurso. Isso é útil quando você deseja atualizar apenas uma parte específica de um recurso, em vez de substituí-lo inteiramente. Exemplo:

```
curl -X PATCH -H "Content-Type: application/json" -d '{"status": "completed"}' http://example.com/resource
```

Isso aplicará uma modificação parcial no recurso especificado, alterando apenas o status para "completado".

TRACE

O método TRACE é utilizado para testar a conectividade entre o cliente e o servidor. Quando um servidor recebe uma requisição TRACE, ele deve retornar a mesma requisição de volta para o cliente. Isso pode ser útil para diagnóstico e depuração de problemas de rede. Exemplo:

```
curl -X TRACE http://example.com/resource
```

CONNECT

Utilizado para estabelecer uma conexão de túnel com um servidor HTTP remoto, temos o CONNECT, comumente usado para estabelecer conexões seguras através de proxies, como em conexões SSL/TLS. Exemplo:

```
curl --proxytunnel -x proxy.example.com:8080 https://example.com
```

Isso estabelecerá uma conexão de túnel através do proxy proxy.example.com:8080 para acessar o recurso <https://example.com>.

LOCK, UNLOCK, PROPFIND, PROPPATCH, MKCOL, COPY, MOVE, DELETE, MKCALENDAR, REPORT, ACL

Esses métodos são parte do protocolo WebDAV (Web-based Distributed Authoring and Versioning), que é uma extensão do HTTP projetada para permitir a edição e gerenciamento de arquivos remotos na web. Eles são utilizados para operações avançadas de edição e controle de versão de recursos na web.

CURL

É uma ferramenta de linha de comando utilizada para transferir dados com URLs. Ele suporta uma ampla variedade de protocolos, incluindo HTTP, HTTPS, FTP, FTPS, SCP, SFTP, LDAP, TFTP e outros, e é amplamente utilizado em ambientes Unix-like, como Linux e macOS, mas também está disponível para Windows e outras plataformas.

Principais recursos do curl

O curl é conhecido por sua versatilidade. Ele pode realizar uma variedade de operações, como fazer solicitações HTTP, enviar e receber dados, fazer upload e download de arquivos, realizar autenticação, entre outras. Além disso, possui uma sintaxe simples e intuitiva, o que o torna fácil de usar, mesmo para iniciantes. Com apenas alguns comandos, é possível realizar diversas operações de transferência de dados.

Por fim, o curl oferece uma ampla gama de opções e parâmetros que permitem personalizar as solicitações de acordo com as necessidades específicas. Por exemplo, é possível enviar cabeçalhos personalizados, definir timeouts, habilitar a verificação SSL, entre outros.

Invoke-RestMethod

O Invoke-RestMethod é um cmdlet do PowerShell que permite fazer solicitações HTTP RESTful de forma fácil e eficiente. Ele simplifica a interação com APIs REST ao encapsular a lógica necessária para fazer solicitações HTTP, enviar e receber dados JSON e lidar com respostas HTTP. O Invoke-RestMethod é especialmente útil para automação e scripting no ambiente do PowerShell.

Principais recursos do Invoke-RestMethod

O Invoke-RestMethod possui uma sintaxe simples e intuitiva que facilita o uso, mesmo para iniciantes no PowerShell. Com apenas um cmdlet, é possível realizar solicitações HTTP e manipular os dados recebidos. Também facilita o envio e recebimento de dados JSON, pois automaticamente serializa e desserializa os dados, permitindo que você trabalhe com objetos PowerShell nativos.

Ele lida automaticamente com as respostas HTTP, permitindo que você acesse os cabeçalhos, o corpo da resposta e outros detalhes importantes da resposta.

Segurança

Autenticação e autorização são duas práticas essenciais para garantir a segurança de uma API Express.js. Veremos uma explicação detalhada sobre cada uma delas.

Autenticação

Autenticação é o processo de verificar a identidade de um usuário ou cliente que está tentando acessar uma aplicação ou recurso. Em uma API, a autenticação geralmente envolve verificar as credenciais do usuário, como nome de usuário e senha, para garantir que ele seja quem diz ser.

Existem várias estratégias de autenticação que podem ser implementadas em uma API Express.js, incluindo:

Autenticação baseada em sessão: o servidor cria uma sessão para o usuário após a autenticação bem-sucedida e envia um cookie de sessão para o cliente. Em cada requisição subsequente, o cliente envia o cookie de sessão para o servidor para autenticação.

Autenticação baseada em token (JWT): o servidor gera um token JWT (JSON Web Token) após a autenticação bem-sucedida e envia-o para o cliente. O cliente inclui o token JWT em cada requisição subsequente no cabeçalho de autorização. O servidor verifica a validade e a autenticidade do token para autorizar o acesso.

Autenticação via API Key: o cliente envia uma chave de API junto com cada requisição para autenticação. O servidor verifica a validade da chave de API para autorizar o acesso.

A escolha da estratégia de autenticação depende dos requisitos de segurança e das necessidades específicas da sua aplicação.

Autorização

É o processo de determinar se um usuário autenticado tem permissão para acessar recursos específicos ou realizar determinadas ações dentro da aplicação. Em uma API, a autorização geralmente envolve verificar as permissões do usuário com base em seu papel ou nível de acesso.

A autorização é implementada após a autenticação bem-sucedida. Depois que o usuário é autenticado, o servidor verifica se possui as permissões necessárias para acessar o recurso solicitado.

A autorização pode ser implementada de várias maneiras, incluindo:

Controle de acesso baseado em função (RBAC): atribuição de permissões com base nas funções do usuário. Por exemplo, um usuário com a função "administrador" pode ter permissões diferentes de um usuário com a função "usuário comum".

Controle de acesso baseado em política (ABAC): atribuição de permissões com base em políticas específicas. Por exemplo, permitir o acesso a um recurso apenas durante determinados horários do dia.

Assim como a autenticação, a escolha da estratégia de autorização depende dos requisitos de segurança e das necessidades específicas da sua aplicação. Ao implementar a autenticação e autorização em sua API Express.js, você pode garantir que apenas usuários autenticados e autorizados tenham acesso aos recursos protegidos, ajudando a manter a segurança e integridade da sua aplicação.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula falamos sobre API e como ela é utilizada com o Express.js. Além disso, abordamos outros métodos que podem ser utilizados para interagir com recursos na web.

EXEMPLO

REFERÊNCIAS

API documentation guide and best practices. [s.a.], [s.d.]. Disponível em: <<https://stoplight.io/api-documentation-guide>>. Acesso em: 13 de maio de 2024.

Criar uma API Web com o Node.js e o Express. Microsoft Learn, [s.d.]. Disponível em: <<https://learn.microsoft.com/pt-br/training/modules/build-web-api-nodejs-express/>>. Acesso em: 13 de maio de 2024.

O que é API? Red Hat, 2023. Disponível em: <<https://www.redhat.com/pt-br/topics/api/what-are-application-programming-interfaces>>. Acesso em: 13 de maio de 2024.

PALAVRAS-CHAVE

Palavras-chave: API, Node.js, Express.

EXEMPLO

POS TECH